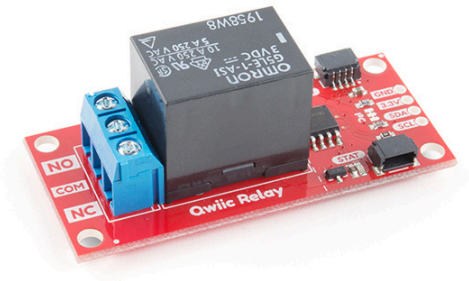
Qwiic Single Relay Hookup Guide

CONTRIBUTORS: *ENGLANDSAURUS*

Introduction

The Qwiic Single Relay is SparkFun's easiest to use relay yet. The single relay can handle up to 5.5A at 240 VAC for long periods of time. The Qwiic connectors and screw terminals also mean that no soldering is necessary.



SparkFun Qwiic Single Relay
COM-15093
$18.42

Product Showcase: SparkFun Qwiic Single and Quad R...

Warning! The Qwiic single relay is great for switching loads like motors, batteries, solenoids, pumps, and more! Keep in mind controlling high voltage (110/220VAC) requires certain precautions. A beginner can do it but if you're unsure then please consider the pre-made IoT Power Relay. It's not I^2C but the IoT Power Relay contains shielding to prevent accidental shock.

Required Materials

To get started, you'll need a microcontroller to control everything. You may not need everything though depending on what you have. Add it to your cart, read through the guide, and adjust the cart as necessary.

SparkFun RedBoard - Programmed with Arduino

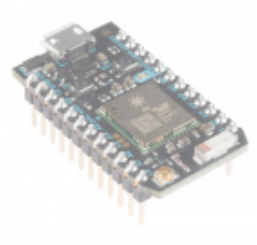
DEV-13975

$22.50

SparkFun ESP32 Thing

DEV-13907

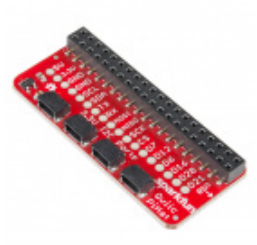
$30.85



Particle Photon (Headers)
WRL-13774
Retired

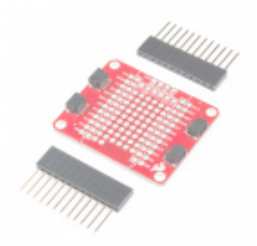
Raspberry Pi 3
DEV-13825
Retired

Now to get into the Qwiic ecosystem, the key will be one of the following Qwiic shields to match your preference of microcontroller:



SparkFun Qwiic HAT for Raspberry Pi
DEV-14459
$6.95

SparkFun Qwiic Shield for Arduino
DEV-14352
$7.75



SparkFun Qwiic Shield for Photon
DEV-14477
Retired

You will also need a Qwiic cable to connect the shield to your Qwiic Single Relay, choose a length that suits your needs.

Qwiic Cable - 100mm
PRT-14427
Retired

Qwiic Cable - 50mm
PRT-14426
Retired

Qwiic Cable - 200mm
PRT-14428
Retired

Qwiic Cable - 500mm
PRT-14429
Retired

Tools

You will need a flush cutter and wire stripper to remove the sheath and insulation from a cable. A Phillips head screwdriver will be required to connect the load's to a screw terminal.

SparkFun Mini Screwdriver
TOL-09146
$1.50

Flush Cutters - Xcelite
TOL-14782
Retired

Self-Adjusting Wire Strippers
TOL-14872
Retired

Suggested Reading

If you aren't familiar with the Qwiic system, we recommend reading here for an overview.

Qwiic Connect System

We would also recommend taking a look at the following tutorials if you aren't familiar with them.

Serial Communication
Asynchronous serial communication concepts: packets, signal levels, baud rates, UARTs and more!

I2C
An introduction to I2C, one of the main embedded communications protocols in use today.

Serial Terminal Basics
This tutorial will show you how to communicate with

Qwiic Shield for Arduino & Photon Hookup Guide

your serial devices using a variety of terminal emulator applications.

Get started with our Qwiic ecosystem with the Qwiic shield for Arduino or Photon.

Hardware Overview

First let's check out some of the characteristics listed in the relay's datasheet that we're dealing with, so we know what to expect out of the board.

Characteristic	Range
Operating Voltage	**1.7V-3.6V**
Supply Current	100mA
Coil Resistance	23.5Ω
I^2C Address	**0x18 (Default)**, *(Jumper changes to 0x19)*
Max Current (Through Relay)	5.5A (240 VAC)

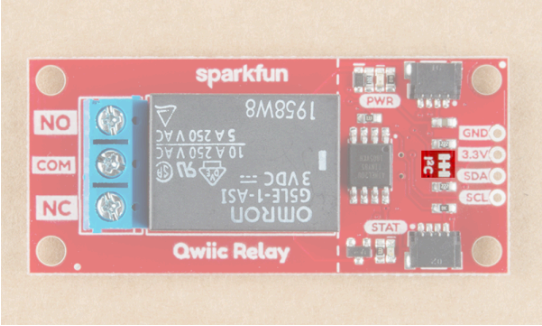
Pins

The following table lists all of the relay's pins and their functionality.

Pin	Description	Direction
GND	Ground	In
3.3V	Power	In
SDA	Data	Bi-directional
SCL	Clock	In
NC	Normally Closed	Switch
NO	Normally Open	Switch
COM	Switch Common	Switch

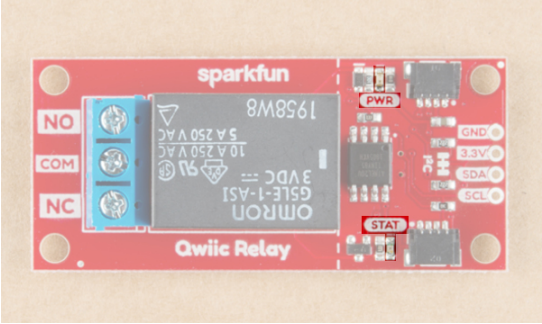
Optional Features

The Qwiic Relay has pull up resistors attached to the I^2C bus; if multiple sensors are connected to the bus with the pull-up resistors enabled, the parallel equivalent resistance will create too strong of a pull-up for the bus to operate correctly. As a general rule of thumb, disable all but one pair of pull-up resistors if multiple devices are connected to the bus. If you need to disconnect the pull up resistors they can be removed by cutting the traces on the corresponding jumpers highlighted below.



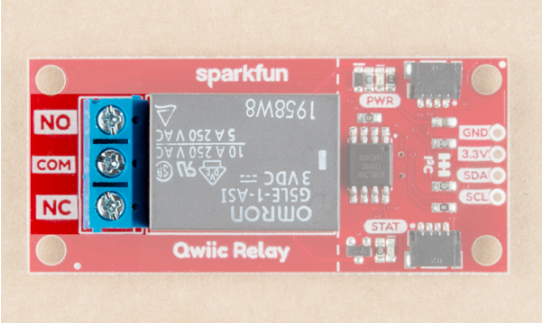
Pull-up Jumper

The Power LED will light up when the board is powered. The Status LED will light up when the relay has been triggered and the switch is closed, both are highlighted in the below image



Power LED

The onboard screw terminal should be used to connect your high-power load, it is highlighted below. The middle `COM` pin should be hooked up to the **Live** wire (Usually black) coming from the wall, while `NO` or `NC` should be connected to the **Live** wire on the device side of things.



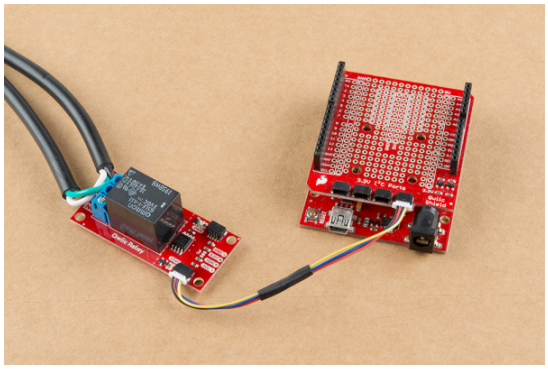
Screw Terminal

Hardware Assembly

If you haven't yet assembled your Qwiic Shield, now would be the time to head on over to that tutorial. Depending on the microcontroller and shield you've chosen, your assembly may be different, but here's a handy link to the Qwiic Shield for Arduino and Photon Hookup Guide to get you started!

Qwiic Shield for Arduino Photon Hookup Guide

With the shield assembled, SparkFun's new Qwiic environment means that connecting the relay could not be easier. Just plug one end of the Qwiic cable into the Qwiic Relay, the other into the Qwiic Shield and you'll be ready to upload a sketch and start turning things on and off. It seems like it's too easy too use, but that's why we made it that way!

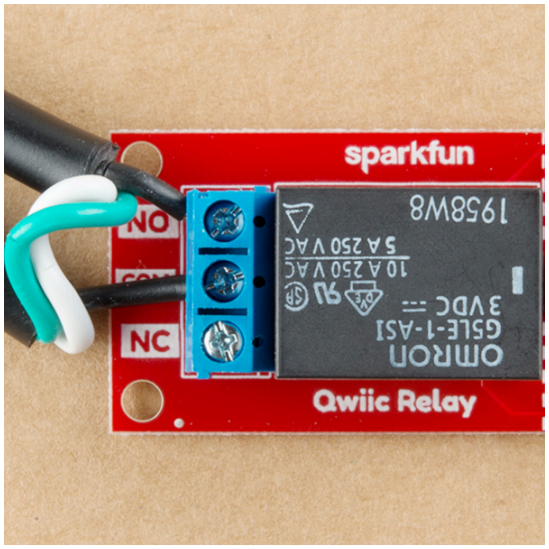


SparkFun RedBoard and Qwiic Shield with the Qwiic Relay attached

Note: Not sure about what color insulation wiring is used in you region? Check out the standard wire insulation colors listed online for reference. If you are unsure about the standard wiring color in your region, please consult a certified electrician to connect to the AC input voltage side.

Warning! Make sure the cable is **not** plugged into the wall as you cut into the wire in the following section.

You'll also need to place the relay in line with the AC powered item you're attempting to control. You'll have to cut your live AC line (usually black or red) and connect one end of the cut wire to `COM` and the other to `NC` or `NO`, depending on what you want the resting state of your device to be. If your AC device is going to be on for most of the time, and you occasionally want to turn it off, you should connect one end to `COM` and the other to `NC`. Connect to `NO` if the device will be off for most of the time. Check out the picture below for a visual aid.



Relay Example Connection

Warning! Make sure that your wires connecting to the wall outlet are secure and are rated to handle the current! Please be careful when handling the contacts when the cable is plugged into a wall outlet. **Touching the contacts while powered could result in injury.**

Looking for information about safety and insulation? Check out the notes about Safety and Insulation from our Beefcake Relay Control Kit.

Each relay takes just over 100 mA to power Daisy chaining these boards together on the same bus will result in pretty large power spikes, you may need a separate 3.3V source depending on what else is included in your project

Arduino Library

We've written a library to make it even easier to get started with the SparkFun Qwiic Single Relay. The library will give you the full functionality of the Qwiic Single Relay without the hub bub of the I^2C data transactions. You can click the link below to download the file or navigate through the Arduino Library Manager by searching **SparkFun Qwiic Relay**. You can also go the Github page and get it directly.

SparkFun Qwiic Relay Arduino Library (ZIP)

This library also works with our Qwiic Quad Relay board. If you need more relays on one PCB, then go check it out!

Note: This example assumes you are using the latest version of the Arduino IDE on your desktop. If this is your first time using Arduino, please review our tutorial on installing the Arduino IDE. If you have not previously installed an Arduino library, please check out our installation guide.

Library Functions Overview

The list below outlines the functions of the Qwiic Relay Arduino Library designed to work with the Qwiic Single Relay along with short descriptions of what they do. The examples cover nearly all of these functions so take a look at them for help integrating them into your own code.

- `bool begin(TwoWire &wirePort = Wire);` - Initialize the Qwiic Single Relay on the I^2C bus
- `float singleRelayVersion();` - Returns the firmware version present on the Qwiic Single Relay.
- `void turnRelayOn();` - Turn the relay on.
- `void turnRelayOff();` - Turn the relay off.

- `void toggleRelay()` - Toggles the relay to the opposite state. The function first checks the status of the relay and is toggled to either `on` or `off` depending on what the status check returns.
- `uint8_t getState();` - Returns the status of the relay. Returns `1` if on or `0` if off.
- `bool changeAddress(uint8_t newAddress);` - Changes the I^2C address of the Qwiic Relay to the value set for `newAddress`. Valid `newAddress` values can be between **0x07** and **0x78**. The new address is written to the memory location in EEPROM that determines the address.

Example Code

The Qwiic Relay is pretty simple, so all of the functions to control it are simply contained in the first example of the SparkFun Qwiic Relay Arduino Library.

Example 1 - Single Relay Basics

Go ahead and unzip the folder to a directory of your choosing and open up `Example1_Single_Relay_Basics`.

Starting at the top, we have `#include`-ed the path to the library's header file as well as to Arduino's I^2C library: `Wire.h`. To use the functions in the SparkFun Qwiic Relay Library we create a version of it, and name it `relay`. You'll notice that in parentheses we have given it the board's address. If you have changed this address or closed the address jumper, than change the `RELAY_ADDR` to *your* address.

```
#include <Wire.h>
#include "SparkFun_Qwiic_Relay.h"

#define RELAY_ADDR 0x18 // Alternate address 0x19



Qwiic_Relay relay(RELAY_ADDR);
```

Let's look at the setup. First we check that we can communicate correctly with the Single Qwiic Relay with the `relay.begin()` function call. If there are some connection issues we'll find out about them here. Note that the code will

not stop if there is an error, so keep an eye out for an eror message. Next we check the version of the firmware that is on your product with the `relay.singleRelayVersion()` function.

```
void setup()
{
  Wire.begin();
  Serial.begin(115200);

  // Let's see
  if(!relay.begin())
    Serial.println("Check connections to Qwiic Rela
y.");
  else
    Serial.println("Ready to flip some switches.");

  float version = relay.singleRelayVersion();
  Serial.print("Firmware Version: ");
  Serial.println(version);
```

Now we turn on and off the relay by using two functions. First the more obvious function `turnRelayOn()` turns on the relay, which can then be turned off with `turnRelayOff`. Next the relay is *toggled* on and off, with the `toggleRelay()` function. The difference here is that the relay is turned off or on depending on the current state of the relay: if **off** ---> turn **on** and vice versa.

```
  // Let's turn on the relay...
  relay.turnRelayOn();
  delay(500);
  // Let's turn that relay off...
  relay.turnRelayOff();
  delay(500);
  // Let's 'toggle' the relay; if it's off turn it on
and vice versa.
  relay.toggleRelay();
  delay(500);
  // Toggle the relay back off.
  relay.toggleRelay();
  delay(500);
```

You're probably wondering why I'm doing this entirely in the setup? In the chance that you're running this for the first time without reading, you won't have a relay constantly turning on and off. Now at the end of the setup we see the final feature of the SparkFun Qwiic Arduino Library.

```
  Serial.print("The Relay is now: ");
  // Is the relay on or off?
  int state = relay.getState();
  if(state == 1)
    Serial.print("On!");
  else if(state == 0)
    Serial.print("Off!");

}
```

Using the `getState` function we can check whether or not the relay is on or off without looking at the board. This can help you programmatically make decisions based on your project's needs.

Resources and Going Further

Now that you've successfully got your Qwiic Single Relay up and running, it's time to incorporate it into your own project!

For more information, check out the resources below:

- Schematic (PDF)
- Eagle Files (ZIP)
- Datasheet (PDF)
- Default Firmware
- SparkFun Qwiic Relay Arduino Library Github Repo
- SparkFun Qwiic Relay Python Package GitHub Repo
- Example Code
- GitHub Repository
- SFE Product Showcase

Need some inspiration for your next project? Check out some of these related tutorials using relays. Be sure to check your current rating when handling the Qwiic Single Relay when browsing some of the other tutorials using relays.

Photon Remote Water Level Sensor

Learn how to build a remote water level sensor for a water storage tank and how to automate a pump based off the readings!

Blynk Board Project Guide

A series of Blynk projects you can set up on the Blynk Board without ever re-programming it.

ESP8266 Powered Propane Poofer

Learn how Nick Poole built a WiFi controlled fire-cannon using the ESP8266 Thing Dev Board!

Blynk Board Bridge Widget Demo

A Blynk project that demonstrates how to use the Bridge widget to get two (or more) Blynk Boards to communicate.

Beefcake Relay Control Hookup Guide

This is a guide for assembling and basic use of the Beefcake Relay Control board

How to Build a Remote Kill Switch

Learn how to build a wireless controller to kill power when things go... sentient.

IoT Power Relay

Using the ESP32 to make a web-configured timed relay.

Qwiic Quad Relay Hookup Guide

SparkFun's Qwiic Quad Relay is a product designed for switching not one but four high powered devices from your Arduino or other low powered microcontroller using I2C.

Or check out these blog posts for inspiration.

Enginursday: Qwiic Escape Room

MARCH 7, 2019

READ POST

Automatic Light Switch on the Internet of Things

MARCH 21, 2019

READ POST

Enginursday: Internet of Terror

JULY 11, 2019

READ POST

Enginursday: Secure DIY Garage Door Opener

JANUARY 16, 2020

READ POST

Enginursday: Keycode Security Starter, Part 2

MARCH 5, 2020

READ POST

Switching to Different Relays

JULY 9, 2020

READ POST